

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ  
ФЕДЕРАЦИИ  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ  
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ  
УНИВЕРСИТЕТ им. В. Г. ШУХОВА» (БГТУ им. В.Г. Шухова)  
Кафедра программного обеспечения  
вычислительной техники и автоматизированных  
систем

**Лабораторная работа №6**  
по дисциплине: Архитектура вычислительных систем  
тема: «**Логические команды и команды сдвига**»

Выполнил: студент группы ВТ-231  
Масленников Д. А.  
Проверили:  
Осипов О.В.  
Бондаренко Т.В.

Белгород 2025

**Цель работы:** изучение команд поразрядной обработки данных.

### Вариант 14

Задания для выполнения к работе

1. Написать программу для вывода чисел на экран согласно варианту задания. При выполнении задания №1 все числа считать беззнаковыми. Написать и использовать функцию `output(a)` для вывода числа `a` на экран или в файл. Функция должна удовлетворять соглашению о вызовах. В функцию для вывода `output` передавать в качестве аргумента переменную размерности 32 или 64 бита, которой достаточно для хранения числа. К примеру, если в задании число указано как 15-разрядное, то аргументом функции должно быть число размером двойное слово, если 40-разрядное, то учетверённое слово. Функция должна выводить столько разрядов числа, сколько указано в задании, даже если старшие разряды равны нулю. Не допускается прямой перебор всех чисел с проверкой, удовлетворяет ли оно условию вывода (за исключением вариантов № 8, 12, 13). Числа выводить в порядке, который является удобным. Проверить количество выведенных чисел с помощью формул комбинаторики. В отчёт включить вывод формул и результаты работы программы.
2. Написать подпрограмму для умножения (`multiplication`) или деления (`division`) большого целого числа на  $2^n$  (в зависимости от варианта задания) с использованием команд сдвига. Подпрограммы должны иметь следующие заголовки: `multiplication(char* a, int n, char* res); division(char* a, int n, char* res)`. Входные параметры: `a` – адрес первого числа в памяти, `n` – степень двойки. Выходные параметры: `res` – адрес массива, куда записывается результат. В случае операции умножения, для массива `res` зарезервировать в два раза больше памяти, чем для множителей `a` и `b`. Числа `a`, `b`, `res` вывести на экран в 16-ричном виде. Подобрать набор тестовых данных для проверки правильности работы подпрограммы.

|    |                                                                                                                                                                                                                                                                                       |                                 |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| 14 | Вывести все 34-разрядные числа, в двоичном представлении которых есть только 2, 3, 4, 5, 6 или 7 подряд идущих единиц, остальные – нули.<br>1: 0000000000 0000000000 0000000000 0011<br>2: 0000000000 0000000000 0000000000 0111<br>...<br>...: 0000000000 0000000000 1111111000 0000 | 45 байт<br>деление<br>со знаком |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|

## Выполнение индивидуального задания:

### Задание 1:

```
section .data
    newline db 10

section .text
    global _start

_start:
    ; Внешний цикл: длина последовательности единиц (len) от 2 до 7
    mov r8, 2      ; r8 = длина (len)

loop_len:
    cmp r8, 7
    jg exit_prog   ; Если len > 7, выход

    ; Создаем базовую маску из r8 единиц.
    ; Формула: (1 << r8) - 1
    mov rcx, r8    ; Подготовка для сдвига
    mov rax, 1
    shl rax, cl     ; 1 << len
    dec rax         ; (1 << len) - 1
    mov r9, rax     ; r9 хранит базовый блок единиц (например, 11, 111...)

    ; Внутренний цикл: сдвиг (shift) от 0 до (34 - len)
    mov r10, 0     ; r10 = текущий сдвиг (shift)

loop_shift:
    mov r11, 34
    sub r11, r8     ; r11 = 34 - len (максимальный сдвиг)
    cmp r10, r11
    jg next_len     ; Если сдвиг больше допустимого, переходим к следующей длине

    ; Формируем число: base_mask << shift
    mov rax, r9     ; Восстанавливаем базовую маску
    mov rcx, r10
    shl rax, cl     ; Сдвигаем на позицию r10

    ; Выводим число в RAX на экран в двоичном виде (34 бита)
    push rax        ; Сохраняем регистры перед вызовом
    push r8
    push r9
    push r10
    push r11

    call print_binary_34

    pop r11
    pop r10
    pop r9
    pop r8
    pop rax

    inc r10         ; shift++
    jmp loop_shift

next_len:
    inc r8          ; len++
    jmp loop_len

exit_prog:
```

[illegible]

## Задание 2:

```
default rel          ; исправляет warning

section .data
    hex_format db "%02X", 0
    newline    db 0xA, 0

    number_a:
        times 44 db 0xAA    ; Младшие 44 байта
        db 0xF0             ; Старший байт (45-й)

    pow_n dd 4              ; Степень сдвига

section .bss
    result_res resb 45

section .text
    global main
    extern printf

main:
    push rbp
    mov rbp, rsp

    ; Подготовка аргументов: RDI=a, ESI=n, RDX=res
    mov rdi, number_a
    mov esi, [pow_n]
    lea rdx, [result_res]    ; Используем LEA для корректной адресации

    call division

    mov rbx, 44              ; Индекс старшего байта

print_loop:
    xor rax, rax
    mov al, [result_res + rbx]

    push rbx
    push rdx                 ; Сохраним rdx, так как printf может его затереть

    mov rdi, hex_format
    mov rsi, rax
    xor rax, rax
    call printf

    pop rdx
    pop rbx

    dec rbx
    jns print_loop          ; Пока rbx >= 0

    ; Перевод строки
    mov rdi, newline
    xor rax, rax
    call printf

    pop rbp
    mov rax, 60              ; sys_exit
    xor rdi, rdi
    syscall

; division(char* a, int n, char* res)
```

```

division:
    push rbx
    push r12

    ; 1. Копирование памяти (a -> res)
    mov rcx, 45
    xor rbx, rbx
copy_loop:
    mov al, [rdi + rbx]
    mov [rdx + rbx], al
    inc rbx
    loop copy_loop

    ; 2. Проверка n=0
    test esi, esi
    jz div_end

    ; 3. Внешний цикл (счетчик сдвигов)
    mov r12d, esi

shift_outer_loop:
    ; --- Обработка старшего байта (индекс 44) ---
    mov rbx, 44
    sar byte [rdx + rbx], 1 ; Знаковый сдвиг. CF устанавливается здесь.

    dec rbx                ; Переход к индексу 43. DEC не меняет CF.

    ; --- Внутренний цикл (индексы 43 -> 0) ---
shift_inner_loop:
    ; [FIX] Убрали CMP. Используем JNS после DEC в конце.

    rcr byte [rdx + rbx], 1 ; Сдвиг через перенос. Берет CF от предыдущего
    байта.

    dec rbx                ; Уменьшаем индекс. CF не меняется.
    jns shift_inner_loop   ; Если индекс >= 0 (Not Sign), продолжаем.

    ; Конец прохода по числу
    dec r12d               ; Уменьшаем счетчик n
    jnz shift_outer_loop   ; Если n > 0, повторяем

div_end:
    pop r12
    pop rbx
    ret

```

Мы делили число на

$2^4$

(сдвиг вправо на 4 бита).

Исходное число (hex): F0 AA AA AA ...

### 1. Старший байт F0 (1111 0000):

- Это отрицательное число (старший бит 1).
- При арифметическом сдвиге (SAR) на 4 бита знаковый бит "размножается".
- $1111\ 0000 \gg 4 = 1111\ 1111 \rightarrow \mathbf{FF}$ .
- (Выдвинутые младшие биты 0000 уходят в следующий байт).

## 2. Следующий байт AA (1010 1010):

- Сверху пришли 4 бита 0000.
- Сдвигаем: 0000 (пришли) + 1010 (были старшими) = 0000 1010 -> **0A**.
- (Выдвинутые младшие биты 1010 (A) уходят в следующий байт).

## 3. Остальные байты AA (1010 1010):

- Сверху пришли 4 бита 1010 (hex A).
- Сдвигаем: 1010 (пришли) + 1010 (были старшими) = 1010 1010 -> **AA**.
- Этот процесс повторяется до конца числа.

Результат работы:

```
~/projects/university/5sem/computer_architecture/lab6 on master !9 ?3 > ./task2
FF0AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

**Вывод:** в ходе выполнения лабораторной работы мы научились совмещать язык программирования СИ с ассемблером, узнали что такое сдвиги, нашли различия между программой и подпрограммой.